MCMM Implementation & Execution Guide

Project: Multi-Voltage Crypto Core (0.95V / 0.75V)

Comprehensive File Breakdown, Execution Steps, and Debugging

This document serves as the master reference for the Multi-Corner Multi-Mode (MCMM) environment configured for the Multi-Voltage Crypto Core in Synopsys IC Compiler II (ICC2). MCMM ensures that the ASIC meets timing, power, and routing constraints across multiple physical operating conditions and functional states.

1. The 6-File MCMM Architecture Breakdown

To prevent the EDA tool from confusing test clocks with functional clocks, and to optimize CPU runtime, the timing constraints are split into four distinct Scenario Constraints (SDC) files, tied together by a Scenario Definition Script, and executed by the Master Run Script.

File 1: `func_ss.sdc` (Functional Mode, Slow Corner)

> **Purpose:** Fixes **Setup (Max) Violations**. Simulates the chip running normal operations at the highest temperature (125°C) and lowest voltage, where silicon resistance is highest and data moves slowest.

```
# Define the fast, functional clock (e.g., 500 MHz -> 2.0ns period)
create_clock -name main_clk -period 2.0 [get_ports clk]

# Set operating conditions to Slow-Slow (ss) 125C
set_operating_conditions -library saed32rvt_ss0p95v125c ss0p95v125c

# Define setup-heavy input/output delays (representing external chip latency)
set_input_delay -max 0.5 -clock main_clk [all_inputs]
set_output_delay -max 0.5 -clock main_clk [all_outputs]

# Clock uncertainty to model jitter and tree skew
set_clock_uncertainty -setup 0.1 [get_clocks main_clk]
```

File 2: `func_ff.sdc` (Functional Mode, Fast Corner)

> **Purpose:** Fixes **Hold (Min) Violations**. Simulates normal operations at extremely low temperatures (-40°C) and highest voltage, where data races through the chip so fast it might overwrite the previous cycle.

```
# Clock remains the same for functional mode
create_clock -name main_clk -period 2.0 [get_ports clk]

# Set operating conditions to Fast-Fast (ff) -40C
set_operating_conditions -library saed32rvt_ff0p95vm40c ff0p95vm40c

# Define hold-heavy input/output delays (minimum delay paths)
set_input_delay -min 0.1 -clock main_clk [all_inputs]
set_output_delay -min 0.1 -clock main_clk [all_outputs]

# Clock uncertainty for hold analysis
set_clock_uncertainty -hold 0.05 [get_clocks main_clk]
```

File 3: `scan_ss.sdc` (Test Mode, Slow Corner)

> **Purpose:** Ensures the scan chains function correctly during manufacturing test without setup violations. Tests the structural shift paths.

```
# Define a slower test clock (e.g., 50 MHz -> 20.0ns period)
create_clock -name main_clk -period 20.0 [get_ports clk]

# Ensure the scan enable pin is held active for shifting
set_case_analysis 1 [get_ports scan_en]

# Slow-Slow Operating Conditions
set_operating_conditions -library saed32rvt_ss0p95v125c ss0p95v125c

# Test equipment delays
set_input_delay -max 2.0 -clock main_clk [get_ports scan_in_1]
set_output_delay -max 2.0 -clock main_clk [get_ports scan_out_1]
```

File 4: `scan_ff.sdc` (Test Mode, Fast Corner)

> **Purpose:** Fixes Hold violations on the scan chains. Since scan flip-flops are connected directly back-to-back, test paths are notoriously prone to hold violations (data shooting through multiple flops in one clock cycle).

```
# Slow test clock
create_clock -name main_clk -period 20.0 [get_ports clk]

# Scan shift mode active
set_case_analysis 1 [get_ports scan_en]

# Fast-Fast Operating Conditions
set_operating_conditions -library saed32rvt_ff0p95vm40c ff0p95vm40c

# Minimum hold timing for the tester
set_input_delay -min 0.5 -clock main_clk [get_ports scan_in_1]
set_output_delay -min 0.5 -clock main_clk [get_ports scan_out_1]
```

File 5: `mcmm_setup.tcl` (Scenario Initialization)

> **Purpose:** This script defines the physical matrix inside ICC2. It merges the modes and corners, loads the 4 SDC files, and activates concurrent analysis.

```
# 1. Create the Scenarios
create_scenario func_ss -mode func -corner ss_125c
create_scenario func_ff -mode func -corner ff_m40c
create_scenario scan_ss -mode scan -corner ss_125c
create_scenario scan_ff -mode scan -corner ff_m40c

# 2. Map the corresponding SDC file to each scenario
read_sdc -scenario func_ss ./inputs/constraints/func_ss.sdc
read_sdc -scenario func_ff ./inputs/constraints/func_ff.sdc
read_sdc -scenario scan_ss ./inputs/constraints/scan_ss.sdc
read_sdc -scenario scan_ff ./inputs/constraints/scan_ff.sdc

# 3. Activate all scenarios for global analysis (Using wildcard fix)
set_scenario_status * -active true
set_scenario_status *ss* -setup true -hold false -leakage_power true
set_scenario_status *ff* -setup false -hold true -leakage_power false
```

File 6: `icc2_master_run.tcl` (Main Execution Flow)

> **Purpose:** The overarching physical design script. It imports the synthesized netlist, loads power intent, builds the floorplan, and initiates standard cell placement.

```
# 1. Load Design and UPF
read_verilog ../outputs/crypto_top_netlist_dft.v
load_upf ../outputs/crypto_top.upf
commit_upf

# 2. Load the MCMM Environment
source ../scripts/mcmm_setup.tcl

# 3. Build the Physical Foundation
source ../scripts/floorplan_mv.tcl
source ../scripts/pg_mesh.tcl

# 4. Standard Cell Placement (Concurrent Optimization)
place_opt

# 5. Load Test Chains and Reorder
read_def -scan ../outputs/crypto_top.scandef
check_scan_chain
optimize_dft -scan
```

2. Execution Protocol

To properly execute the flow from scratch in ICC2 without holding legacy data in memory, follow these terminal steps:

1. **Launch ICC2:** Open your Linux terminal, navigate to the `run/` directory, and type `icc2_shell`.
2. **Verify Data:** Ensure the updated `.v`, `.upf`, and `.scandef` files generated from `dc_shell` are in your `outputs/` or `data/` folder.
3. **Source the Run Script:** Execute `source ../scripts/icc2_master_run.tcl`.
4. **Verify MCMM Load:** Run `report_scenario`. Ensure the table shows `Active = true` for all 4 scenarios, with `ss` scenarios tracking Setup/Power and `ff` scenarios tracking Hold.

3. Common ICC2 Violations & Resolutions

Error / Warning Code	Description & Meaning	Resolution Strategy
(CMD-007) Required argument missing	Occurs when using `-all` instead of `*` in `set_scenario_status`. ICC2 strict syntax requires collections or wildcards for scenario lists.	Change `set_scenario_status -all` to `set_scenario_status *`.
(UIC-034) Redefining clock 'main_clk'	The tool notices multiple `create_clock` commands for the same port across different SDC files.	**Ignore.** This is expected and correct behavior in an MCMM flow, as clocks are isolated within their own memory scenarios.

(NEX-018) / **(TIM-102)** No valid parasitic for corner	The timing extractor cannot find the `.tluplus` file containing the physical metal layer physics (Resistance/ Capacitance).	For lab/learning environments, safely **ignore**. The tool will default to Zero Wire-Load models. In production, load the TLU+ file using `read_parasitic_tech`.
(DFT-015) optimize_dft terminated due to unplaced cell	Attempting to reorder scan chains based on physical proximity before the standard cells have been given physical X/Y coordinates on the floorplan.	Ensure `place_opt` completes fully before running `optimize_dft -scan`.
(DEFR-065) Cannot find cell/ port in design	Name mismatch between the Verilog netlist and the SCANDEF file (usually due to brackets `[]` vs underscores `_`).	Run `change_names -rules verilog -hierarchy` in Design Compiler **before** generating the `.scandef`.